

Loops & functions

The essential rules needed to understand the rest of the notebook.

Working in Colab

Run cells **from top to bottom**. Editing a cell does not execute it, and variables keep values from previous runs. Before retrying a mutation exercise, rerun the cell that creates the original list.

For loops

BASIC LOOP

```
for i in range(3):  
    print(i)      # prints 0, 1, 2 on separate lines
```

RULE	WHAT TO REMEMBER
range(n)	Produces 0 to n - 1; n is excluded .
Indentation	Everything inside the indented block repeats.

Functions

DEFINE AND CALL A FUNCTION

```
def f(x, y):  
    a = x + 1  
    return a + y  
  
result = f(1, 2)    # result is 4
```

TERM	MEANING
Parameters / arguments	The definition uses parameters x, y; the call supplies arguments 1, 2.
Local variable	a is created within the function call.
return	Sends the result back to the caller and ends the function.
print()	Displays a value; it is not the same as returning it.

One detail worth remembering

A function with no explicit return returns None. Colab may show the final expression of a cell automatically; that does not mean return itself prints.

Lists, tuples & sorting

Focus on indexing and whether an action changes an existing object.

Lists versus tuples

LIST	TUPLE
<code>a = ["a", "b", "c"]</code>	<code>t = ("a", "b", "c")</code>
Mutable: stored items can be replaced.	Immutable: items cannot be replaced.

Indexing: read an item

SAME INDEXING RULES APPLY TO TUPLES

```
a = ["a", "b", "c"]
a[0]    # "a" (first)
a[1]    # "b" (second)
a[-1]   # "c" (last)
a[-2]   # "b" (second from the end)
```

Indices start at 0. For n items, the last positive index is $n - 1$; negative indexing starts at -1 .

Mutability: change an item

READ WITH `A[0]`; CHANGE WITH `A[0] = ...`

```
a[0] = "z"      # works; a is now ["z", "b", "c"]
# t[0] = "z"    # TypeError: tuples reject item assignment
```

Sorting: the distinction that matters

OPERATION	CHANGES a?	RESULT
<code>b = sorted(a)</code>	No	A new sorted list is stored in <code>b</code> .
<code>a.sort()</code>	Yes	Sorts <code>a</code> in place ; returns <code>None</code> .

MINI EXAMPLE

```
a = [4, 2, 3, 1]
b = sorted(a)    # b: [1, 2, 3, 4]; a unchanged
a.sort()         # a: [1, 2, 3, 4]
```

Revision question

When you see an operation on a list, ask whether it **modifies the existing list** or **creates/assigns another one**. This becomes essential for the last part of the notebook.

Simple Matplotlib plots

The notebook uses ordinary Python lists to build x and y coordinates.

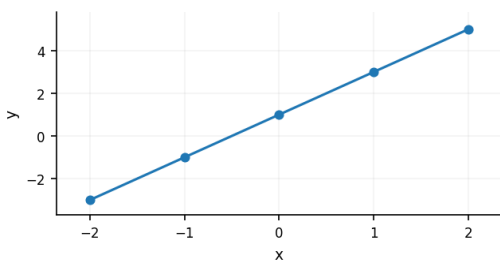
Essential setup

Every x-value needs a matching y-value: the coordinate lists must have **equal length**. Call `plt.show()` after all graph commands.

ONE COMPLETE PLOT

```
import matplotlib.pyplot as plt

x_plot = [-2, -1, 0, 1, 2]
y_plot = [-3, -1, 1, 3, 5]      # y = 2*x + 1
plt.plot(x_plot, y_plot)        # joined line segments
plt.scatter(x_plot, y_plot)     # individual dots
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



Expected: an increasing line through (0, 1), with five sampled points.

Only four plotting commands to remember

COMMAND	PURPOSE
<code>plt.plot(x, y)</code>	Connect consecutive supplied points.
<code>plt.scatter(x, y)</code>	Show the supplied points as dots.
<code>plt.xlabel()</code> / <code>plt.ylabel()</code>	Label the horizontal and vertical axes.
<code>plt.show()</code>	Display the finished figure.

Notebook exercise: quadratic data

CREATE VALUES FOR Y = X SQUARED

```
x_plot = [-2, -1, 0, 1, 2]
y_plot = []
for value in x_plot:
    y_plot.append(value**2)
# y_plot = [4, 1, 0, 1, 4]
```

Remember: Reset `y_plot = []` before re-running. Five quadratic points joined by `plt.plot()` make straight segments, not an exact smooth curve.

Mutation, reassignment & scope

This distinction explains most of the notebook's prediction questions.

Mental model

A variable name refers to an object. Passing a list to a function makes the parameter refer to the **same list object** - it does not automatically create a copy.

Change the list versus change the name

INSIDE THE FUNCTION	EFFECT ON CALLER'S LIST
<code>values[0] = 99</code>	Changes it: updates the shared list object.
<code>values = [2, 3]</code>	Does not replace it: reassigns the local variable.

EARLIER MUTATION REMAINS AFTER REASSIGNMENT

```
a = [1, 4, 9]
def change(values):
    values[0] = 99    # shared list is modified
    values = [2, 3]   # local name now refers elsewhere

change(a)
print(a)             # [99, 4, 9]
```

Loop trap: index versus element

THE SECOND LOOP ONLY REASSIGNS AN INTEGER NAME

```
a = [1, 4, 9]
for i in range(len(a)):
    a[i] += 2        # a becomes [3, 6, 11]

a = [1, 4, 9]
for element in a:
    element += 2     # a stays [1, 4, 9]
```

Local versus global names

A function can access and mutate a global list if that name has **no local binding**. But if the function also assigns to the same name later, Python treats the name as local throughout that function. Reading it first raises `UnboundLocalError`.

NOTEBOOK CASE (J): ERROR HAPPENS ON THE CALL

```
example_list = [1, 4, 9]
def wrong(input_list):
    example_list[0] = 9    # unassigned local: error
    example_list = [1, 2]
# wrong(example_list)     # uncomment to see the error
```

What to test yourself on

Use the original notebook for the full exercises; this is the compact reference.

Question 1: print a list in reverse

ONE ELEMENT ON EACH OUTPUT LINE

```
def print_reverse(a):
    for i in range(len(a)):
        print(a[-(i + 1)])

print_reverse([1, 4, 9, 16, 25]) # 25, 16, 9, 4, 1
```

Negative indices go -1, -2, etc. The + 1 matters because -0 is the same as 0.

Question 2: prediction cases (a)-(j), grouped by idea

Each original block begins with its own [1, 4, 9, 16, 25]. Try predicting before reading these outcomes.

CASE(S)	WHAT THE CODE DOES	RESULT / EFFECT
a	Reassign the outer name.	[1, 2, 3, 4]
b, d	Update list items by index (directly / inside function).	[3, 6, 11, 18, 27]
c	Add 2 to each loop variable, not an indexed item.	Original list unchanged.
e, i	Reassign a name inside the function.	Outer list unchanged.
f, g	Mutate index 0, then reassign the local parameter.	[99, 4, 9, 16, 25]
h	Mutate the global list.	[99, 4, 9, 16, 25]
j	Access a local name before assigning to it.	UnboundLocalError

Final five-point checklist

- Explain why `range(3)` gives 0, 1, 2, not 3.
- Distinguish `return` from `print()`.
- Use positive/negative indexing and explain why tuples reject item assignment.
- Explain `sorted(a)` versus `a.sort()` and `plot` versus `scatter`.
- Predict the difference between **mutating an object** and **rebinding a name**.

Highest-value revision idea

When reading code, ask: **(1) Which object does this name refer to? (2) Does this line modify that object, or only reassign the name?**